ONLINE STORE · VEHICLE WORKFLOW GUIDE

Part 7

The branch tooling: the Web Export Prep add-on and the demo site

Reference for the two things this guide ships: a Blender add-on that turns the TSC master scene into a web .glb plus manifest in one click, and the `store-demo/` folder that is a working landing page. How each was tested, how to extend them, and what to script next.

In this part

1. Where the web branch sits in the toolkit
2. Installing and using the add-on
3. What the add-on reads: the tags
4. What the add-on does, step by step
5. What the add-on writes: the manifest
6. The demo site, file by file
7. How it was tested
8. Extending the viewer
9. Further scripts worth writing
10. Known limits

Files: `tools/blender/web_export_prep.py`, `store-demo/`, `store-demo/test/build_demo.py`, `store-demo/test/drive.js`. Tested with Blender 4.2's Python module and Chromium with WebGL (**demo**).

1 · Where the web branch sits in the toolkit

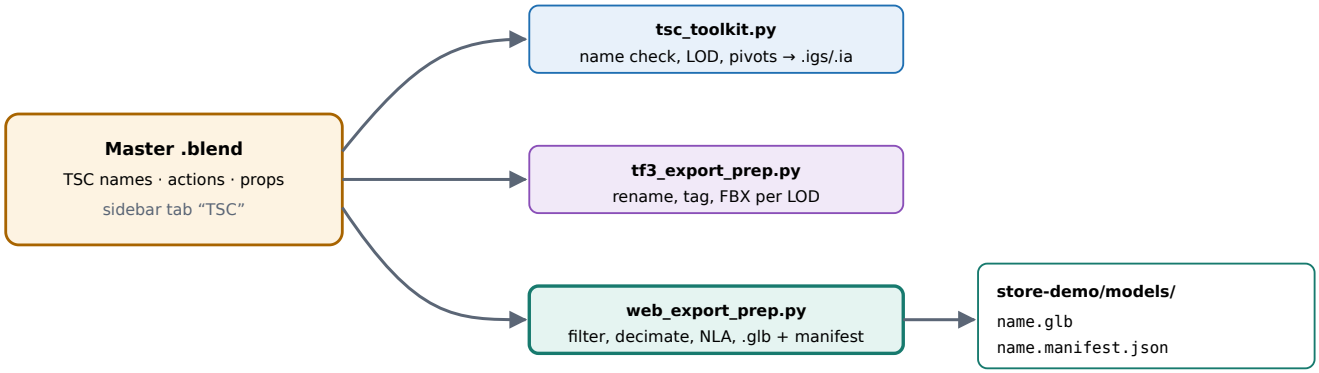


Figure 7.1 · Three panels, one tab. All three add-ons register under the same “TSC” sidebar tab, read the same object names and actions, and never change the master except to push actions into NLA strips (which TSC export tolerates and TF3 export needs anyway).

2 · Installing and using the add-on

1. Blender ▸ Edit ▸ Preferences ▸ Add-ons ▸ **Install…** ▸ pick `tools/blender/web_export_prep.py` ▸ tick it. (Same as the other two.)
2. In the 3D viewport press `N`, open the **TSC** tab, panel **Web viewer**.
3. **Output folder:** your site's `models/`. **Model name:** file stem (`little_joe`). **Triangle budget** 60,000. **Max texture px** 2048.
4. Optionally set two scene custom properties (Scene ▸ Custom Properties): `web_title` and `web_blurb`; they become the panel text on the page.
5. Press **Export web GLB + manifest**. The status bar shows triangles and hotspot count; the full report is in the Text Editor as `Web_Export`.
6. Add `models/little_joe.manifest.json` to `MODEL_POOL` in `index.html`; run the local server (Part 5 §1); click things.

3 · What the add-on reads: the tags

Property	On	Value	Effect
`web_anim`	Object	clip name, e.g. `open_door_l1`	The object's action is pushed to an NLA strip of that name; the object becomes a clickable hotspot playing that clip. Several objects may share a name (two pantograph arms = one clip).
`web_label`	Object	text, e.g. `Door L1`	Hover label and button text. If absent, the clip name is prettified (“Open door l1”). The first label given for a clip is used for every object of that clip.

Property	On	Value	Effect
`web_hotspot`	Object (mesh or empty)	clip name	Object has no animation of its own but clicking it plays the clip: a door frame for a door leaf, a pantograph base for its arms, a large invisible box over a thin part.
`web_skip`	Object	1	Left out of the web model even if it would otherwise be included.
`web_glass`	Material	1	Material blend mode set to Alpha Blend before export so windows come through as transparent.
`web_title` , `web_blurb`	Scene	text	Manifest title and blurb.

Dropped automatically: objects whose name's LOD digit is not 1 (`2_…` , `3_…`), names containing `shadow_` , `primarydigits` , `tsc_coupling` , `tsc_driver` , and empties that carry no web tag. Lights, cameras and armature-less bones are not exported.

4 · What the add-on does, step by step

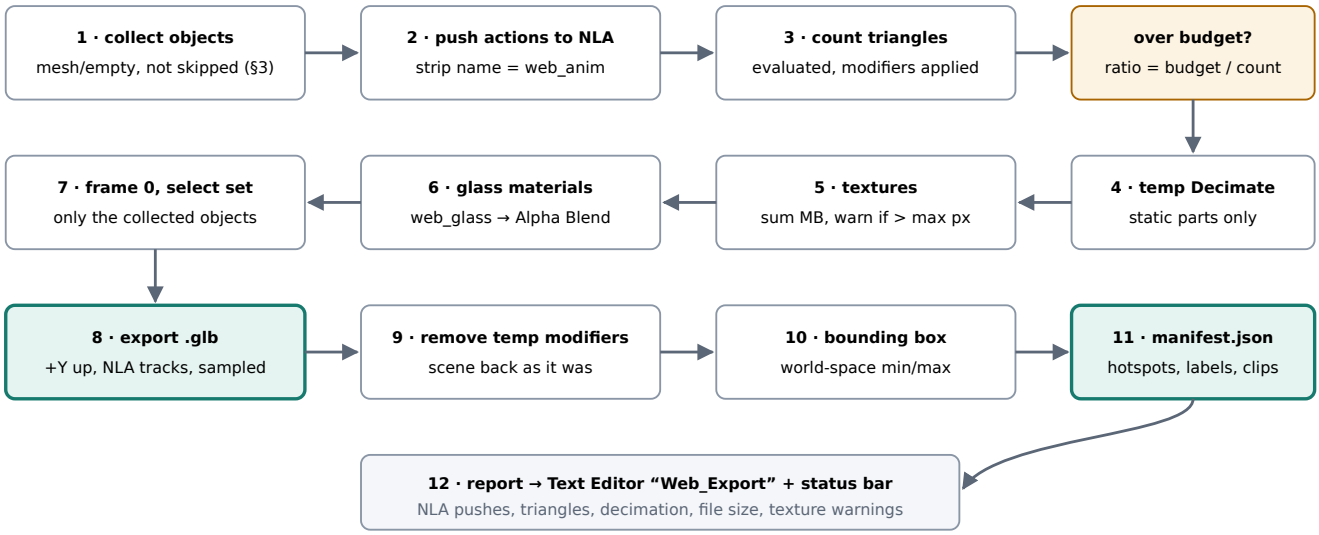


Figure 7.2 · The operator, as a flow. Steps 4 and 9 are why the scene is untouched afterwards: the decimation is a temporary modifier on non-hotspot meshes only, added before export and removed after.

The glTF exporter call, so you can match it by hand or change one flag:

```
bpy.ops.export_scene.gltf(
    filepath=path, export_format="GLB", use_selection=True, export_yup=True,
    export_apply=True, export_animations=True, export_nla_strips=True,
    export_frame_range=False, export_force_sampling=True,
    export_image_format="AUTO", export_texcoords=True, export_normals=True,
    export_materials="EXPORT", export_draco_mesh_compression_enable=False)
```

A console line saying the Draco library was not found is harmless: compression is off.

5 · What the add-on writes: the manifest

```
{
  "model": "sample_electric.glb",            // relative to the manifest
  "title": "Class EF-4 \"Little Joe\" Heavy Electric (demo stand-in)",
  "blurb": "Demo model built from primitives …",
  "triangles": 816,
  "bbox": [[-1.5, -9.0, 0.0], [1.5, 9.025, 4.16]],   // Blender metres, Z up
  "textureMB": 0.0,
  "animations": ["open_door_l1", "open_door_l2", "open_door_r1", "open_door_r2",
                 "raise_panto_front", "raise_panto_rear", "wipers"],
  "hotspots": [
    { "node": "1_0500_door_l1",        "label": "Door L1",          "animations":
["open_door_l1"] },
    { "node": "1_0128_panto01_base",   "label": "Pantograph front", "animations":
["raise_panto_front"] },
    { "node": "1_0128_panto01_part01", "label": "Pantograph front", "animations":
["raise_panto_front"] },
    …
  ]
}
```

It is plain text. You can edit a label, add a second clip to a hotspot's `animations` list (a door that also lights its step), or write a manifest by hand for a model exported some other way. The viewer only needs `model`, `title` and `hotspots`.

6 · The demo site, file by file

File	Edit?	Role
`index.html`	yes, text	Landing page: header, hero (stage + panel), catalogue cards, footer. The import map and `MODEL_POOL` live in its head.
`css/site.css`	yes, colours	Dark theme; the five colours are variables at the top. Phone layout under 900 px.
`js/viewer.js`	rarely	The viewer (Part 4). Exposes `window.viewer`: `load(url)`, `trigger(node)`, `manifest`, `ready`, `screenPos(node)`.
`vendor/`	no	three.js 0.170 (MIT): `three.module.js`, `jsm/loaders/GLTFLoader.js`, `jsm/controls/OrbitControls.js`, `jsm/utils/BufferGeometryUtils.js`, licence file.
`models/`	add	Your exports. The demo pair is the primitive loco.
`test/build_demo.py`	reference	Builds the primitive loco in Blender, tags it, and runs the add-on headless. A worked example of every tag.
`test/drive.js`	reference	Opens the page in headless Chromium, hovers and clicks hotspots, takes the screenshots used in Part 4, reports page errors.

7 · How it was tested

1. **Model:** `test/build_demo.py` built a body, roof, four windows (`web_glass`), four doors with origins on their hinge edges and an opening key at frame 24, two three-axle bogies, two pantographs (lower arm `web_anim`, upper arm the same name, base `web_hotspot`), two wipers, plus a `shadow_` mesh and a `tsc_coupling` empty that must be dropped.
2. **Export report:** 10 actions pushed to NLA strips; 816 triangles (budget 60,000, no decimation); `sample_electric.glb` 0.09 MB; manifest with 12 hotspot entries and 7 animations. The shadow mesh and coupling empty were not in the .glb.
3. **Browser:** `test/drive.js` loaded the page over the local server, waited for `viewer.ready`, hovered door R1 (label shown), clicked it (door open, button lit), clicked the pantograph *base* (arms rose: the `web_hotspot` path), pressed the "Door L1" button, and took the five screenshots in Part 4.
4. **Result:** status line "816 triangles · 7 animations · click a part"; seven buttons (one per label, so each pantograph's three nodes gave one button); three marked active; **no page errors**.

To repeat it after a change: start the server in `store-demo/`, then `node store-demo/test/drive.js` with Playwright on the Node path.

8 · Extending the viewer

Want	Change
A viewer on each product page	Copy `index.html`, keep the hero, set `MODEL_POOL` to that vehicle's manifest, replace the cards with the spec and copy from Part 2.
Camera presets ("cab", "bogie")	Add buttons that set `camera.position` and `controls.target`; store the positions in the manifest as a `views` list.
Sound on click	In `trigger()`, `new Audio('sfx/door.ogg').play()`; put the file name in the hotspot entry.
Livery switch	Export the body with two materials, or swap the base-colour texture at run time (`TextureLoader`) when a swatch is clicked. Keep real railroad schemes out (Part 6 §6).
Wheels turning while idle	Export a `wheels` clip and give it `LoopRepeat` in `load()` instead of `LoopOnce`; start it automatically.
Night mode	Lower the hemisphere light, raise an emissive on lamp materials (tag `web_emissive`, add to the add-on in ten lines).
Draco compression	Enable it in the exporter and copy three's `jsm/libs/draco/` into `vendor/`; set `loader.setDRACOLoader(...)`. Halves mesh bytes.
Smaller library download	Point the import map at a CDN copy of three (cached across sites) instead of `vendor/`.

9 · Further scripts worth writing

Script	What it would save
Hotspots from TSC control values	Read the `.ia` or the blueprint's animation names and propose `web_anim` tags for every animated object, so a loco already set up for the game needs no manual tagging.
Batch export the catalogue	Loop over a folder of master .blend files headless (as `build_demo.py` does) and regenerate every `models/` pair in one command, then deploy with the host's CLI.
Product thumbnail renderer	Place the camera as `frameModel()` does, render an Eevee turntable of 8 frames per vehicle for the catalogue cards and the store listing.
Texture packer for the web	Reuse the TF3 packer: albedo to JPEG, normal to PNG, roughness = 1 − gloss, cap at 2048 px, report MB per vehicle.
Manifest validator	Open the .glb (a glTF is JSON + binary) and check that every hotspot node and animation name exists; run before upload.
Spec-sheet generator	From the TSC blueprint values (mass, power, max speed, brake type) write the spec table of the product page so copy and sim never disagree.

10 · Known limits

- The add-on exports objects, not armatures: parts animated by bones (valve gear with an armature) export as skinned meshes and play, but hotspot tags must be on the mesh object, not on bones.
- Shape-key animations export; constraint-driven motion (a pantograph upper arm following the lower via a constraint) is sampled and exports correctly, but only if the constraint target is also exported.
- Decimation is uniform per object by ratio; parts that must keep a silhouette should be tagged as hotspots or given `web_skip` and replaced by a hand-made low version.
- Labels are per clip, not per object, by design (two arms, one button). Give two doors two clips if they need two buttons.
- The demo page is not a store: catalogue cards are placeholders to be replaced by links or embeds from the provider (Part 5 §5).
- three.js is pinned at 0.170; a newer copy may rename an addon path. Update all of `vendor/` together.